

Implementação em Hardware do Gauss-Jordan com Pivotamento Parcial

Ponto fixo Q8.8, matriz aumentada $[A \mid I \mid b]$ — módulo `matrix_inv_gj`

Estudo de apoio — `07.Studies/inversao_de_matrizes`

Agosto de 2026

1 Escopo e relação com a revisão teórica

O documento `matrizes-revisao-algebra_linear-v00.pdf` justifica **por que** Gauss-Jordan com pivotamento parcial foi escolhido como o primeiro algoritmo de `matrix_inv` a ser implementado (melhor equilíbrio entre custo aritmético e regularidade de controle para hardware, entre os quatro métodos estudados). Este documento é o complemento prático: registra **como** esse algoritmo foi de fato implementado nos três níveis do projeto — referência Python (`03.Reference/matrix_inv/algorithms/gauss_jordan.py`), RTL (`04.RTL/matrix_inv/matrix_inv_gj.v` e `common/divider_q88.v`) e testbench (`05.Sim/matrix_inv/tb_matrix_inv_gj.v`) — incluindo as decisões de ponto fixo, a arquitetura do datapath e um exemplo numérico completo, passo a passo, idêntico ao que circula pelo hardware real.

As decisões de arquitetura resumidas aqui (formato numérico, interface, epsilon) estão registradas com mais detalhe em `02.Architecture/open_decisions.md`.

2 Definições

2.1 Formato Q8.8

Todo dado de entrada/saída do módulo (elementos de A , b , x e A^{-1}) é representado em ponto fixo **Q8.8**: uma palavra de 16 bits em complemento de dois, com 8 bits inteiros (incluindo o sinal) e 8 bits fracionários. O valor real de uma palavra r (inteiro de 16 bits, `raw`) é:

$$\text{valor}(r) = \frac{r}{2^8} = \frac{r}{256}$$

Faixa representável: $[-128, 127,99609375]$. Resolução (1 LSB): $1/256 \approx 0,0039$.

Internamente, o produto de dois valores Q8.8 (dois números de 16 bits) é calculado sem truncamento intermediário: o resultado bruto de uma multiplicação de dois valores de 16 bits cabe em 32 bits e está naturalmente no formato **Q16.16**. Esse valor de 32 bits só é arredondado e saturado de volta para Q8.8 no momento em que é escrito em um registrador de I/O (ver Seção 2.5). Essa é a técnica padrão de “guard bits” internos citada em `02.Architecture/open_decisions.md`.

2.2 Matriz aumentada $[A \mid I \mid b]$

Ao invés de calcular A^{-1} e depois multiplicar por b separadamente, o hardware resolve os dois de uma só vez: a matriz de entrada é aumentada horizontalmente com a matriz

identidade I (para obter A^{-1} como subproduto natural da eliminação) e com o vetor b (para obter x na mesma passada). Para uma matriz $N \times N$, a matriz aumentada tem:

$$M = 2N + 1 \text{ colunas}$$

Por exemplo, para $N = 2$: $M = 5$. As operações de linha do Gauss-Jordan são aplicadas às M colunas simultaneamente; ao final, o bloco que começou como I vira A^{-1} e a última coluna vira x .

2.3 Êpsilon de singularidade

Um pivô é tratado como “zero” (matriz singular ou mal condicionada demais para o formato Q8.8) quando seu módulo cai abaixo de um limiar fixo:

$$\varepsilon_{Q8.8} = 4 \text{ LSB} = 0x0004 \approx 0,0156$$

Quando isso ocorre em qualquer coluna, o hardware aborta a eliminação e sinaliza a saída `singular=1`; o conteúdo de `x_out/ainv_out` nesse caso é indefinido pela especificação da interface (zerado por conveniência de simulação). Um efeito colateral de ponto fixo documentado durante a Sprint 1: uma matriz *matematicamente* singular nem sempre deixa um pivô exatamente zero após a eliminação, porque a composição divisão→multiplicação em Q8.8 não é uma inversa exata (ver Seção 2.5) — na prática o resíduo cai quase sempre abaixo de $\varepsilon_{Q8.8}$, mas não há garantia formal disso para qualquer entrada.

2.4 Divisor iterativo Q8.8

A divisão do passo de normalização (linha do pivô $\div$ pivô) é feita por um divisor sequencial *shift-subtract* (restoring, radix-2), não pelo operador $/$ do Verilog (que esconderia o custo real de área/latência atrás de uma IP genérica do Quartus — ver 02.Architecture/open_decisions.md). Dados dois valores Q8.8 a e b (com $b \neq 0$, garantido pelo teste de $\varepsilon_{Q8.8}$ antes de iniciar a divisão), o quociente é:

$$q = \text{ sinal}(a) \cdot \text{ sinal}(b) \cdot \left\lfloor \frac{|a| \cdot 2^8}{|b|} \right\rfloor, \quad \text{saturado em } [-32768, 32767]$$

O hardware calcula isso com um registrador de resto de $\text{WORD_BITS} + \text{FRAC_BITS} + 1 = 25$ bits, consumindo o dividendo estendido ($|a| \ll 8$, 24 bits) um bit por ciclo: **24 ciclos** por divisão, sempre (a contagem de ciclos não depende dos valores, só da largura de palavra — ver `divider_q88.v`).

2.5 Eliminação linha-paralela e o arredondamento do MAC

Depois de normalizar a linha do pivô k , cada outra linha i é atualizada em uma única passada, processando as M colunas em paralelo (em vez de uma divisão/multiplicação por vez):

$$\text{linha}_i \leftarrow \text{linha}_i - \text{fator}_i \cdot \text{linha}_k, \quad \text{fator}_i = \text{linha}_i[k] \text{ (antes da atualização)}$$

Cada elemento passa por **duas** saturações Q8.8, não uma:

1. o produto $\text{fator} \times \text{linha}_k[j]$ (Q16.16, exato, sem perda) é arredondado para o mais próximo e saturado para Q8.8 — essa é a “ δ ” subtraída;

2. a subtração $\text{linha}_i[j] - \delta$ é saturada para Q8.8 de novo, pois o resultado pode extrapolar a faixa mesmo com os dois operandos dentro dela.

Esta é exatamente a função `eliminate_col` do RTL e o par `widen_mul/rescale_acc_to_q88` do modelo Python — os dois implementam a mesma sequência de arredondamento e saturação, bit a bit.

2.6 Interface e handshake

O módulo expõe `start` (pulso) / `busy` / `done` (pulso) / `singular`, com A , b , x e A^{-1} em barramentos `flatten` (não portas de array, que exigiriam SystemVerilog — ver 04.RTL/matrix_inv/README.md para o motivo). Um divisor único (Seção 2.4) é compartilhado por todas as colunas e chamado sequencialmente; a eliminação (Seção 2.5) usa M multiplicadores para atualizar uma linha inteira por vez — essa combinação (“divisor serial + eliminação linha-paralela”) foi uma escolha explícita de compromisso entre área e latência, registrada em 02.Architecture/open_decisions.md.

3 Resolução passo a passo

O algoritmo percorre as colunas $k = 0, \dots, N-1$ da submatriz A ; cada coluna passa pelos seguintes estados da FSM (`matrix_inv_gj.v`):

1. **Carregamento** (`S_LOAD`, uma vez só): monta a matriz aumentada $[A \mid I \mid b]$ a partir dos barramentos de entrada.
2. **Busca de pivô** (`S_PIVOT_SEARCH`): entre as linhas ainda não processadas ($i \geq k$), escolhe a de maior $|\text{valor}|$ na coluna k (pivotamento parcial). Se $|\text{pivô}| < \varepsilon_{Q8.8}$ (Seção 2.3), a operação termina com `singular=1`.
3. **Troca de linhas** (`S_PIVOT_SWAP`, só se necessário): move a linha do pivô escolhido para a posição k .
4. **Normalização** (`S_NORM_ISSUE/S_NORM_WAIT`): divide cada uma das M colunas da linha k pelo valor do pivô, uma coluna por vez, usando o divisor serial (Seção 2.4). Ao final, a posição (k, k) vale exatamente 1,0 (0x0100) — autodivisão em Q8.8 é exata, sem arredondamento (Seção 2.5 não se aplica aqui, só à eliminação).
5. **Eliminação** (`S_ELIM_ROW`): para cada linha $i \neq k$, aplica a atualização linha-paralela da Seção 2.5, zerando (ou aproximando de zero) a coluna k dessa linha.
6. **Próxima coluna** (`S_NEXT_COLUMN`): $k \leftarrow k + 1$; repete a partir do passo 2 até $k = N$.
7. **Extração do resultado** (`S_DONE`): as colunas $[N, 2N)$ da matriz aumentada são A^{-1} ; a última coluna é x . Pulso de `done`.

4 Exemplo passo a passo

Este exemplo usa exatamente os valores do teste de regressão `tb_matrix_inv_gj.v` para $N = 2$ (bit a bit idêntico ao que passa pelo simulador). Todas as matrizes abaixo são mostradas em decimal e, entre parênteses, em Q8.8 bruto (`raw`).

4.1 Entrada

$$A = \begin{pmatrix} 4,0 & 1,0 \\ 2,0 & 3,0 \end{pmatrix} \quad b = \begin{pmatrix} 1,0 \\ 2,0 \end{pmatrix}$$

Em Q8.8 raw: $A = \begin{pmatrix} 1024 & 256 \\ 512 & 768 \end{pmatrix}$, $b = \begin{pmatrix} 256 \\ 512 \end{pmatrix}$.

Matriz aumentada inicial $[A \mid I \mid b]$ ($M = 5$ colunas):

$$\begin{pmatrix} 1024 & 256 & 256 & 0 & 256 \\ 512 & 768 & 0 & 256 & 512 \end{pmatrix} = \begin{pmatrix} 4,0 & 1,0 & 1,0 & 0,0 & 1,0 \\ 2,0 & 3,0 & 0,0 & 1,0 & 2,0 \end{pmatrix}$$

4.2 Coluna $k = 0$

Busca de pivô: candidatos na coluna 0: $|1024|$ (linha 0) vs. $|512|$ (linha 1). Linha 0 vence, sem necessidade de troca. $|1024| \geq \varepsilon_{Q8.8}$: segue.

Normalização (divide a linha 0 inteira por 1024, 5 divisões seriais de 24 ciclos cada):

$$(1024 \ 256 \ 256 \ 0 \ 256) / 1024 \rightarrow (256 \ 64 \ 64 \ 0 \ 64) = (1,0 \ 0,25 \ 0,25 \ 0,0 \ 0,25)$$

Eliminação da linha 1 (fator = $512 = 2,0$, a Seção 2.5 aplicada às 5 colunas em paralelo):

$$\delta = \text{round_sat}(2,0 \times (1,0, 0,25, 0,25, 0,0, 0,25)) = (512, 128, 128, 0, 128) = (2,0, 0,5, 0,5, 0,0, 0,5)$$

$$\begin{aligned} \text{linha}_1 &\leftarrow (512, 768, 0, 256, 512) - (512, 128, 128, 0, 128) = (0, 640, -128, 256, 384) \\ &= (0,0, 2,5, -0,5, 1,0, 1,5) \end{aligned}$$

Matriz aumentada após $k = 0$:

$$\begin{pmatrix} 256 & 64 & 64 & 0 & 64 \\ 0 & 640 & -128 & 256 & 384 \end{pmatrix} = \begin{pmatrix} 1,0 & 0,25 & 0,25 & 0,0 & 0,25 \\ 0,0 & 2,5 & -0,5 & 1,0 & 1,5 \end{pmatrix}$$

4.3 Coluna $k = 1$

Busca de pivô: só resta a linha 1 (640). Sem troca.

Normalização (linha 1 $\div 640$):

$$\begin{aligned} (0 \ 640 \ -128 \ 256 \ 384) / 640 &\rightarrow (0 \ 256 \ -51 \ 102 \ 153) \\ &= (0,0 \ 1,0 \ -0,19921875 \ 0,3984375 \ 0,59765625) \end{aligned}$$

(note a divisão $-128/640 = -0,2$ exato em ponto flutuante vira $-0,19921875$ em Q8.8: a truncção da divisão perde 0,00078125, menos de meio LSB.)

Eliminação da linha 0 (fator = $64 = 0,25$):

$$\begin{aligned} \delta &= \text{round_sat}(0,25 \times (0,0, 1,0, -0,19921875, 0,3984375, 0,59765625)) \\ &= (0, 64, -13, 26, 38) = (0,0, 0,25, -0,05078125, 0,1015625, 0,1484375) \\ \text{linha}_0 &\leftarrow (256, 64, 64, 0, 64) - (0, 64, -13, 26, 38) = (256, 0, 77, -26, 26) \end{aligned}$$

4.4 Resultado final

$$\begin{pmatrix} 256 & 0 & 77 & -26 & 26 \\ 0 & 256 & -51 & 102 & 153 \end{pmatrix} = \begin{pmatrix} 1,0 & 0,0 & 0,30078125 & -0,1015625 & 0,1015625 \\ 0,0 & 1,0 & -0,19921875 & 0,3984375 & 0,59765625 \end{pmatrix}$$

Lendo as colunas $[2, 4)$ como A^{-1} e a última como x :

$$A_{\text{hw}}^{-1} = \begin{pmatrix} 0,30078125 & -0,1015625 \\ -0,19921875 & 0,3984375 \end{pmatrix} \quad x_{\text{hw}} = \begin{pmatrix} 0,1015625 \\ 0,59765625 \end{pmatrix}$$

Comparação com a solução exata (ponto flutuante, $\det A = 10$):

$$A_{\text{exato}}^{-1} = \begin{pmatrix} 0,3 & -0,1 \\ -0,2 & 0,4 \end{pmatrix} \quad x_{\text{exato}} = \begin{pmatrix} 0,1 \\ 0,6 \end{pmatrix}$$

O maior desvio é 0,00234375 (no elemento $A_{1,0}^{-1}$: $-0,2$ vs. $-0,19921875$), pouco menos de 1 LSB ($1/256 = 0,00390625$) — consistente com a única fonte de erro deste exemplo ser a truncção (sem arredondamento) do divisor descrita na Seção 2.4, já que nenhum valor intermediário saturou.

4.5 Custo em ciclos

Para $N = 2$ ($M = 5$): 2 colunas $\times$ 5 divisões $\times$ 24 ciclos = 240 ciclos só de divisão, mais busca de pivô, troca (quando ocorre) e eliminação (poucos ciclos cada). Medido em simulação (`tb_matrix_inv_gj.v`): **280 ciclos** do pulso de `start` ao pulso de `done` para este caso — a divisão domina o custo, exatamente como antecipado na revisão teórica (`matrizes-revisao-algebra_linear-v00.pdf`, Seção 4.1).

5 Validação

Este exemplo é um caso manual do mesmo processo executado pela suíte de regressão da Sprint 1: 36 casos gerados por `03.Reference/tools/gen_matrix_inv_vectors.py` (bem-condicionados, quase-singulares e singulares, para $N = 2, 3, 4$), todos conferidos bit a bit contra o modelo de referência Python pelo testbench Verilog — 36/36 passando. Detalhes de como rodar: `03.Reference/README.md` (`make test`) e `05.Sim/matrix_inv/README.md` (`iverilog/vvp`).